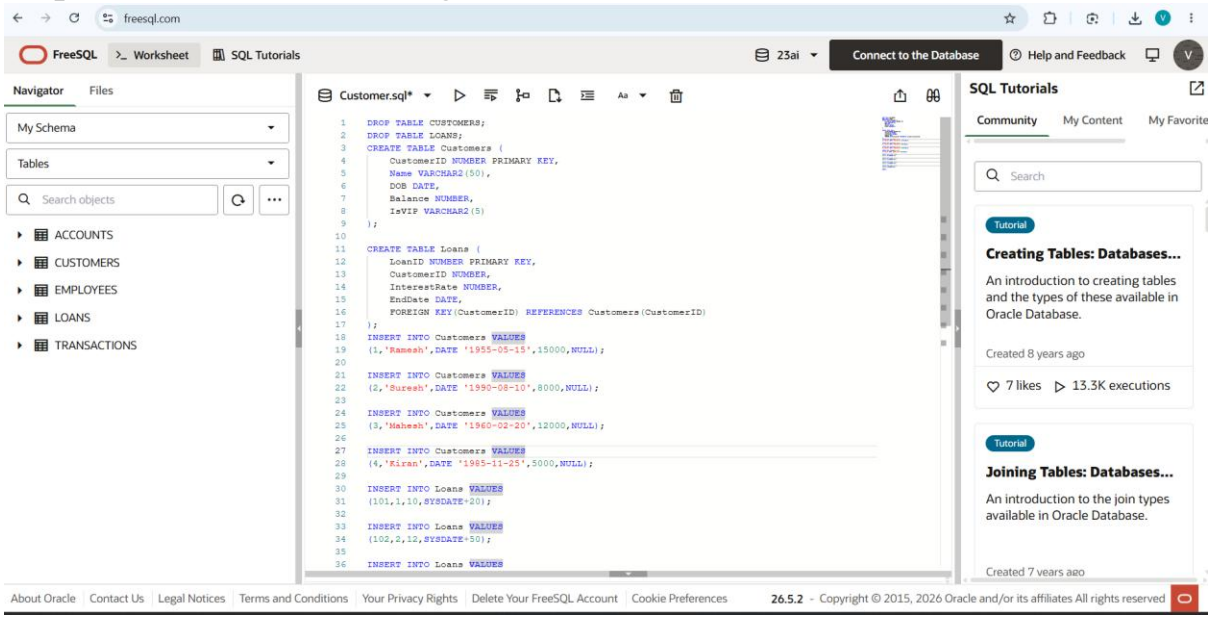
Exercise 1: Control Structures in PL/SQL

Step 1: Table created values get inserted



Step 2:

Scenario 1

Problem Statement:

Apply a 1% discount to loan interest rates for customers above 60 years old.

Explanation:

This PL/SQL block retrieves all customers from the Customers table and calculates their age using the DOB (Date of Birth) field. It then loops through each customer record and checks whether the customer's age is greater than 60 years. If the condition is satisfied, the corresponding loan's interest rate in the Loans table is reduced by 1%. After processing all customers, the COMMIT statement permanently saves the updated interest rates in the database. A success message is displayed using DBMS_OUTPUT.PUT_LINE to confirm that the discount has been applied successfully.

```
SELECT * FROM Loans;

DECLARE
    v_age NUMBER;
BEGIN

    FOR cust IN (
        SELECT CustomerID, DOB
        FROM Customers
    ) LOOP

        v_age := FLOOR(
                    MONTHS_BETWEEN(
                    SYSDATE,
                    cust.DOB
                    ) / 12);

        IF v_age > 60 THEN

            UPDATE Loans
            SET InterestRate =
                InterestRate - 1
            WHERE CustomerID =
                    cust.CustomerID;

        END IF;

    END LOOP;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE(
    'Interest rate discount applied successfully.');

END;
```

OUTPUT:

	LOANID	CUSTOMERID	INTERESTRATE	ENDDATE
1	101	1	10	7/12/2026, 1:42:20
2	102	2	12	8/11/2026, 1:42:20
3	103	3	11	7/2/2026, 1:42:20 P
4	104	4	9	9/20/2026, 1:42:20

Step 3:

Scenario 2

Problem Statement:
Set **IsVIP = TRUE** for customers whose balance is greater than 10,000.

Explanation:
This PL/SQL block retrieves all customer records from the **Customers** table

and checks each customer's account balance. It loops through the records one by one using a **FOR LOOP**. If a customer's balance is greater than **10,000**, the **IsVIP** column is updated to **TRUE**; otherwise, it is updated to **FALSE**. After processing all customer records, the **COMMIT** statement permanently saves the changes in the database. A success message is displayed using **DBMS_OUTPUT.PUT_LINE** to indicate that the VIP status has been updated successfully.

```
BEGIN

FOR cust IN (
SELECT CustomerID, Balance
FROM Customers
)
LOOP

  IF cust.Balance > 10000 THEN

     UPDATE Customers
     SET IsVIP = 'TRUE'
     WHERE CustomerID = cust.CustomerID;

  ELSE

     UPDATE Customers
     SET IsVIP = 'FALSE'
     WHERE CustomerID = cust.CustomerID;

  END IF;

END LOOP;

COMMIT;

DBMS_OUTPUT.PUT_LINE('VIP Status Updated Successfully');

END;
/
SELECT CustomerID,
       Name,
       Balance,
       IsVIP
FROM Customers
  ORDER BY CustomerID;
  SELECT CustomerID, Balance, IsVIP
  FROM Customers;
```

Output:

	CUSTOMERID	NAME	BALANCE	ISVIP
1	1	Ramesh	15000	TRUE
2	2	Suresh	8000	FALSE
3	3	Mahesh	12000	TRUE
4	4	Kiran	5000	FALSE

Step 4:

Scenario 3:
Problem Statement:
Send reminder messages for loans that are due within the next 30 days.

Explanation:
This PL/SQL block retrieves loan records whose EndDate falls between the current date (SYSDATE) and the next 30 days. It joins the Customers and Loans tables to obtain customer names along with their loan details. The block then loops through each matching record and displays a reminder message using DBMS_OUTPUT.PUT_LINE, showing the customer's name, loan ID, and due date. This helps the bank proactively notify customers about upcoming loan repayments and avoid missed due dates.

```
SELECT
    c.Name,
    l.LoanID,
    TO_CHAR(l.EndDate,'DD-MON-YYYY') AS Due_Date
FROM Customers c
JOIN Loans l
ON c.CustomerID = l.CustomerID
WHERE l.EndDate BETWEEN SYSDATE AND SYSDATE + 30;
```

Output:

	NAME	LOANID	DUE_DATE
1	Ramesh	101	08-JUL-2026
2	Mahesh	103	02-JUL-2026

103

Exercise 3 – Stored Procedures in PL/SQL

Step 1: Scenario 1

Problem Statement

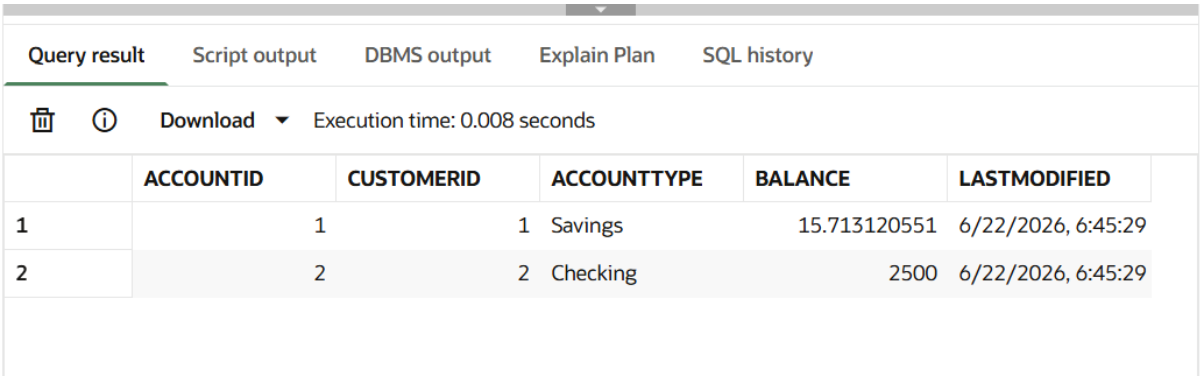
Calculate and credit 1% monthly interest to all savings accounts.

Explanation:

This stored procedure automatically calculates monthly interest for every account present in the SavingsAccounts table. The balance of each account is increased by 1% of its existing value using an UPDATE statement. Once all account balances are updated, the COMMIT command saves the modifications permanently. A confirmation message is displayed after successful execution, indicating that the monthly interest has been credited.

```
1   CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
2   AS
3   BEGIN
4
5       UPDATE Accounts
6       SET Balance = Balance + (Balance * 0.01)
7       WHERE AccountType = 'Savings';
8
9       COMMIT;
10
11  END;
12  /
13  BEGIN
14      ProcessMonthlyInterest;
15  END;
16  /
17  SELECT * FROM Accounts;
18
```

Output:



Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.008 seconds

	ACCOUNTID	CUSTOMERID	ACCOUNTTYPE	BALANCE	LASTMODIFIED
1	1	1	Savings	15.713120551	6/22/2026, 6:45:29
2	2	2	Checking	2500	6/22/2026, 6:45:29

Step 2: Scenario 2

Problem Statement

Update employee salaries by providing a department name and bonus percentage as input parameters.

Explanation

This stored procedure increases the salaries of employees belonging to a specific department. It accepts the department name and bonus percentage as parameters and calculates the revised salary by applying the given bonus. After updating the records, the transaction is committed to save the changes permanently in the Employees table.

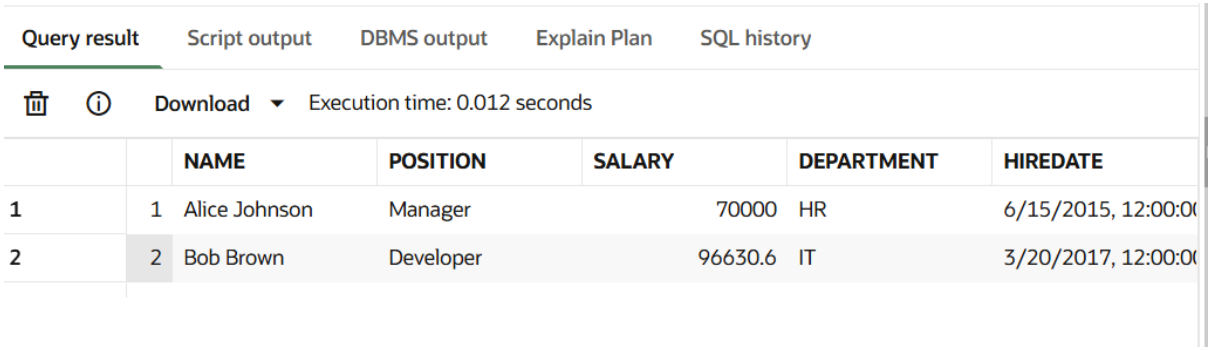
```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
    p_department IN VARCHAR2,
    p_bonus IN NUMBER
)
AS
BEGIN

    UPDATE Employees
    SET Salary = Salary + (Salary * p_bonus / 100)
    WHERE Department = p_department;

    COMMIT;

END;
/
BEGIN
    UpdateEmployeeBonus('IT',10);
END;
/
SELECT * FROM Employees;
```

Output:



Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.012 seconds

		NAME	POSITION	SALARY	DEPARTMENT	HIREDATE
1	1	Alice Johnson	Manager	70000	HR	6/15/2015, 12:00:0(
2	2	Bob Brown	Developer	96630.6	IT	3/20/2017, 12:00:0(

```
SQL> BEGIN
        UpdateEmployeeBonus('IT',10);
     END;



PL/SQL procedure successfully completed.

Elapsed: 00:00:00.008
```

Step 3: Scenario 3

Problem Statement

Transfer a specified amount from one account to another only if sufficient balance is available in the source account.

Explanation

This stored procedure enables secure fund transfers between customer accounts. It first checks whether the source account contains enough balance to complete the transaction. If sufficient funds are available, the amount is deducted from the sender's account and credited to the receiver's account. Otherwise, the transfer is cancelled and a message is displayed. The transaction is committed only after a successful transfer.

```
CREATE OR REPLACE PROCEDURE TransferFunds(
    p_from_account IN NUMBER,
    p_to_account IN NUMBER,
    p_amount IN NUMBER
)
AS
    v_balance NUMBER;
BEGIN

    SELECT Balance
    INTO v_balance
    FROM Accounts
    WHERE AccountID = p_from_account;

    IF v_balance >= p_amount THEN

        UPDATE Accounts
        SET Balance = Balance - p_amount
        WHERE AccountID = p_from_account;

        UPDATE Accounts
        SET Balance = Balance + p_amount
        WHERE AccountID = p_to_account;

        COMMIT;

    END IF;
```

```
    
END;
/
BEGIN
    TransferFunds(1,2,5);
END;
/
SELECT * FROM Accounts;
```

Output:

```
SQL> BEGIN
         TransferFunds(1,2,5);
     END;



PL/SQL procedure successfully completed.

Elapsed: 00:00:00.014
```